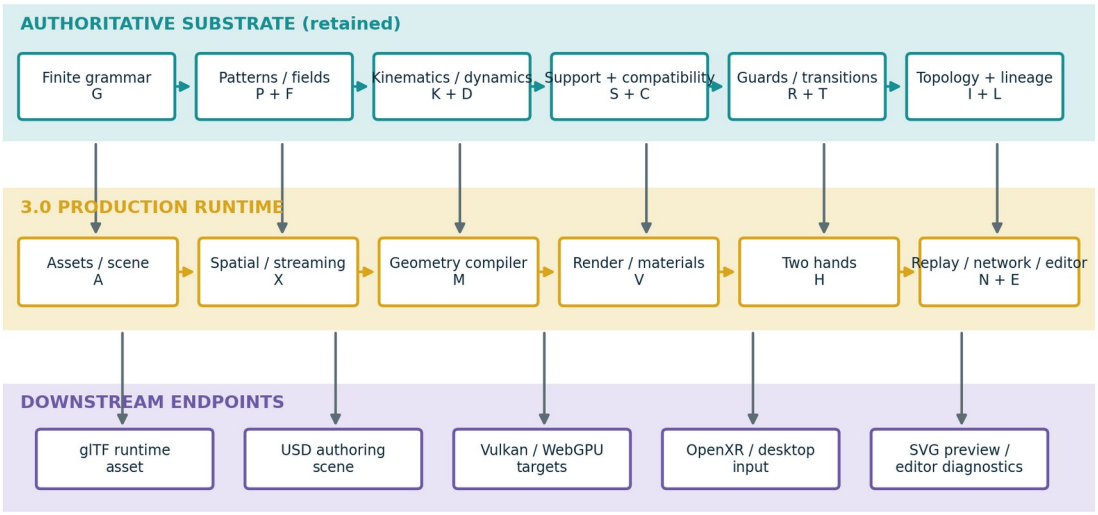
UGTS-KC TWO HANDS 3.0

Interactive Graphics, Game Runtime and Content Substrate

Production companion and tested reference vertical slice

UGTS-KC Two Hands 3.0 architecture

The 2.0 query-first core remains authoritative; production layers compile, interact, replay and render downstream.



Discrete change remains: support -> compatibility -> guard -> verified proposal -> deterministic commit -> lineage/replay.

Prepared for	Tom Klootwijk
Requester-supplied identifier	NL200678942
Requester-supplied date of birth	10 July 1990
Document date / version	16 August 2026 / 3.0.0

ATTRIBUTION BOUNDARY

The name, identifier and date of birth above were supplied by the requester and were not independently verified. This technical package is not legal proof of identity, authorship, ownership, priority or patentability.

Contents and reading rule

READING RULE

The core is still not a renderer. Geometry and kinematics define typed state and relations; spatial structures prune candidates; hand, GPU and game systems may propose interactions; only verified deterministic commit changes authoritative state and lineage. Rendering and export remain downstream projections.

Document control

Item	Status / count	Meaning
UGTS source-normalized baseline	197 mechanisms	M001-M197 retained by source/reference.
UGTS-KC 2.0 additions	60 mechanisms	M198-M257 pattern, field, topology, kinematics and dynamics.
KC Two Hands 3.0 additions	72 mechanisms	M258-M329 production runtime and interaction expansion.
Extended catalog total	329 mechanisms	Baseline plus the two explicitly separated engineering additions.
Executable verification	117 tests PASS	47 retained KC 2.0 tests plus 70 new KC 3.0 tests.
Physical-GPU claim	false	No physical Vulkan GPU benchmark is claimed by this package.
Runtime dependency	Python standard library	Matplotlib/CairoSVG are used only to regenerate report diagrams.

Immediate source basis

Source	SHA-256 prefix	Treatment
UGTS-0 architecture PDF	c991aba24cc939ba...	Referenced by hash; not redistributed.
UGTS-GN 1.1 GPU-native addendum	3a6b6c57118ca4e2...	Referenced by hash; not redistributed.
UGTS-KC 2.0 package	834997b0139ee43...	Immediate implementation baseline; not nested in this ZIP.
UGTS-KC 2.0 report	a29dd9959ebd300c...	Immediate technical companion baseline.

1. Executive upgrade decision

3.0 changes the center of gravity from mathematical breadth to a usable graphics/game vertical slice.

1.1 Baseline retained

The mature UGTS line remains a query-first relation and event substrate. A finite grammar and typed state define what exists; local support and conventional spatial bounds remove irrelevant candidates; compatibility determines whether supported states may couple; guards and numerical status define candidate events; deterministic transition logic updates scene, route, topology, dynamics and lineage. KC Two Hands 3.0 does not replace that sequence with a frame loop, renderer or physics engine.

UGTS-KC 2.0 already supplied the needed mathematical breadth: parameterized pattern families, implicit fields, topology descriptors, SE(2)/SE(3) kinematics, moving frames, jerk-bounded timing, bounded dynamics and event-degeneracy handling. The remaining production gap was the path from those operators to scenes, meshes, materials, hand interaction, replay and export.

1.2 What is new

- A versioned asset/scene layer with stable node identity, content hashes, transform hierarchies, layers, units and migration history.
- A deterministic geometry compiler for adaptive cubic curves, strokes, swept tubes and bounded marching-tetrahedra field meshing.
- A hybrid spatial layer containing AABB, BVH, ray/frustum queries, streaming cells, interest sets and culling reason codes.
- Typed PBR preview materials, an acyclic procedural material graph, color transforms and a deterministic CPU/SVG renderer.
- Typed left/right hand state, pinch hysteresis, bimanual translation/scale/alignment/twist, cooperative grab and handover lineage.
- A fixed-step event runtime with proposal verification, deterministic conflict resolution, snapshots, checkpoints, rollback and divergence detection.
- glTF 2.0 JSON, USDA, UGTS JSON, replay JSON and SVG output adapters, plus Vulkan/WebGPU/OpenXR production contracts.

1.3 Upgrade decision

DECISION - GO

GO for the 3.0 production companion because it demonstrates one complete path from 2.0 pattern and field definitions to compiled geometry, scene composition, two-hand interaction, authoritative event commit, replay and downstream graphics export. The release remains a reference vertical slice, not a claim of a finished commercial engine.

1.4 Concrete deliverables

Deliverable	Location	Status
Companion report	report/*.pdf and *.docx	Delivered and visually verified.
M258-M329 catalog	spec/kc3_mechanisms_*	72 additions in CSV and JSON.
Combined M198-M329 catalog	spec/engineering_catalog_*	132 engineering entries; baseline M001-M197 retained by reference.
Production schema and contracts	spec/*.json and *.md	Supplied; example validates.
Reference runtime	src/ugts_kc3/	Dependency-free Python implementation.
Retained KC 2.0 oracle	src/ugts_kc/	Copied for a self-contained test baseline.
Sandbox vertical slice	examples/two_hands_sandbox.py	Executed; outputs captured.
Shaders	shaders/*.wgsl and *.glsl	Layout/preview prototypes; not physical-GPU benchmarks.

Deliverable	Location	Status
Verification	tests/ and validation/	117 passing tests plus hashes and manifests.

2. Continuity and evidence discipline

The 3.0 report distinguishes inherited architecture, engineering implementation and future adapter targets.

2.1 Source-derived continuity

UGTS-0 defines the coherent core as finite grammar, typed state, local support, compatibility, event solving, transition, lineage and an external novelty log. Its game integration section explicitly allows conventional meshes, rigid bodies, audio, animation, editor tools and rasterization to coexist with the authoritative query substrate. That separation is the basis of this 3.0 companion.

UGTS-GN 1.1 makes the canonical order even more explicit: local support, compatibility, guard crossing, verified event, route/transition and lineage. It also establishes an important evidence boundary: software Vulkan execution or intermediate code is not the same as measured physical-GPU advantage. KC Two Hands 3.0 retains that distinction.

2.2 Engineering additions

Mechanisms M258-M329 are engineering additions. They are not described as if they were extracted verbatim from the source PDFs. Each catalog row records a normalized definition, formula or contract, integration role and validation status. Fully implemented reference mechanisms are separated from shader contracts, metadata targets, specified boundaries and acceptance gates.

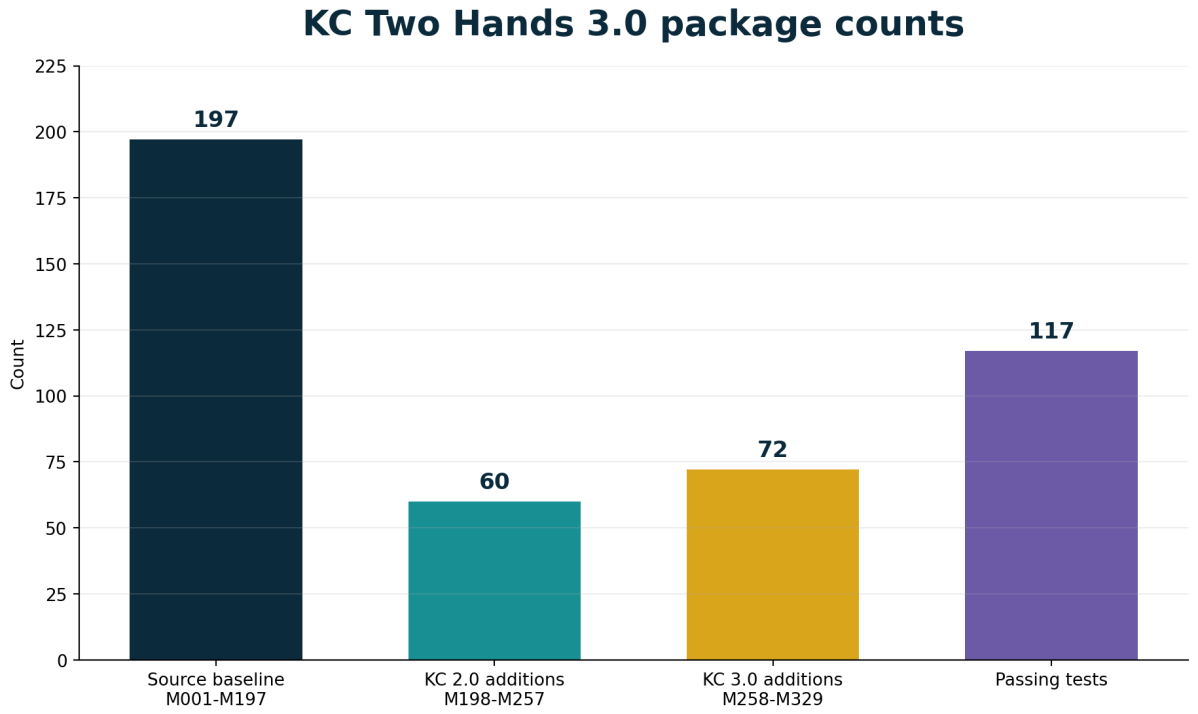


Figure 2. Mechanism and test counts. The 3.0 addition is explicitly separated from the source-normalized baseline and the 2.0 engineering layer.

2.3 Claim boundary

- No universal O(1) world or arbitrary exact event solver is promoted.
- No zero-memory, zero-latency, zero-heat or one-bit-complete-state claim is introduced.
- Topology remains explicit gluing/routing and never substitutes for contact mechanics or conservation laws.

- Generated meshes and pixels are downstream representations unless promoted under a validated application contract.
- Vulkan, WebGPU, OpenXR, OpenUSD, MaterialX and OpenColorIO are integration targets; full conformance is not claimed.
- A named physical GPU, driver, workload, precision, equal-error baseline and measured latency/energy are required before a GPU performance claim is promoted.

3. Formal KC Two Hands 3.0 architecture

Production layers are added around - not inside - the authoritative event core.

3.1 Extended definition

```
UGTS-KC2 = (G, P, F, K, D, R, S, C, T, I, L)
UGTS-KC3 = UGTS-KC2 + (A, X, M, V, H, N, E)

A : assets, instances, scene layers and migration
X : bounds, BVH, culling, streaming and interest sets
M : curve/field-to-mesh compilation and error contracts
V : materials, color, rendering and interchange outputs
H : left/right hand input and bimanual interaction
N : event sequence, checkpoint, rollback and replay
E : diagnostics, editor inspection and validation gates
```

The notation does not imply that every field belongs in one hot record. A production implementation should keep stable IDs, transform and event-critical state compact while leaving mesh payloads, material graphs, editor metadata and source definitions in referenced cold storage.

3.2 Canonical production path

```
pattern / field + kinematic state
    -> conventional bounds / spatial query
    -> local support pruning
    -> topology / mode / policy compatibility
    -> guard value + derivative + error interval
    -> event proposal
    -> deterministic verification and conflict policy
    -> scene / topology / dynamic patch
    -> lineage + replay / novelty log
    -> optional render, export, audio or haptics
```

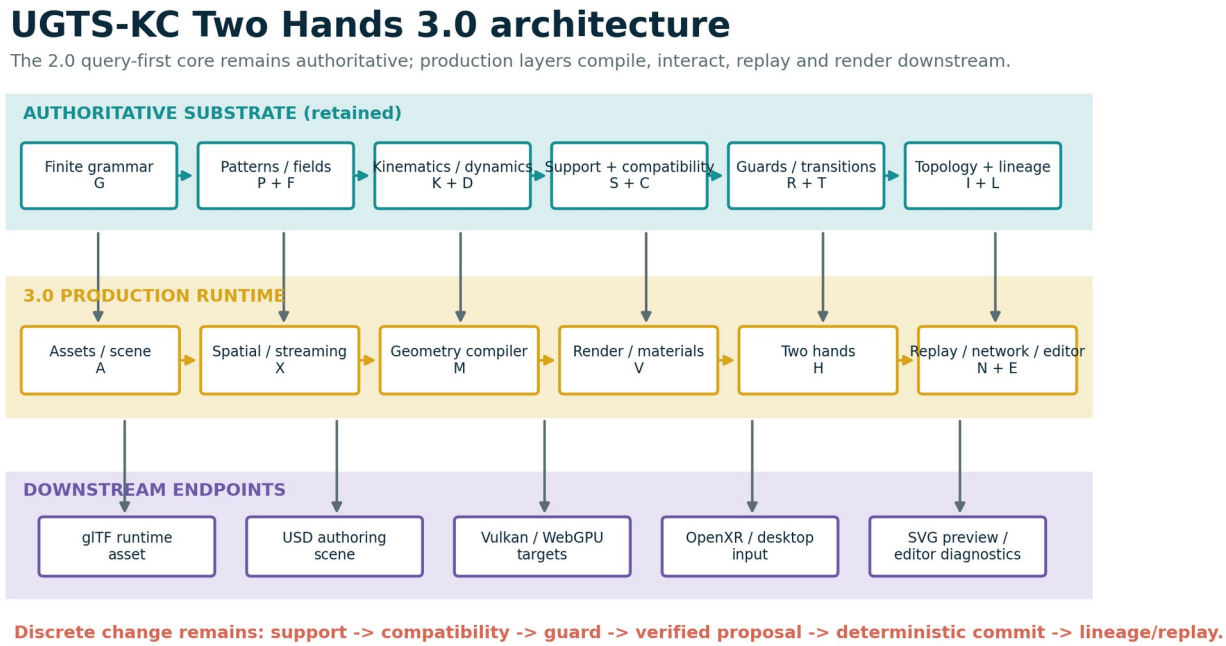


Figure 3. The retained substrate remains authoritative. Scene, spatial, geometry, render, hand and replay layers form a production-facing shell.

3.3 State and identity split

```
q_hot = (position, time, phase, sheet, orientation,
         branch, policy, uncertainty, velocity, mode,
         node_id, lineage_ref)

node_cold = (asset_id, parent_id, local_transform, bounds,
             material_binding, interaction_binding,
             layer, tags, editor_metadata)
```

Node identity, asset identity and lineage are distinct. Multiple nodes may instance the same asset; multiple nodes may share a coordinate; a node may change asset or parent through an event without losing its identity. Content hashes support cache and integrity checks, but they do not replace lineage or legal identity.

3.4 Minimum 3.0 query set

Query	Result / contract
world_transform(node)	Composed parent-to-local transform with cycle rejection.
scene_bounds()	Finite union of visible node bounds or an explicit empty result.
query_aabb / query_ray / query_frustum	Conventional coarse candidates with deterministic ordering.
interest_set(center,radius,tags)	Per-user/client replication and streaming scope.
compile_curve / compile_field	Derived mesh plus approximation metadata.
hand_transform(anchor,left,right)	Bimanual transform or bounded rejection state.
commit_proposals(proposals)	Verified ordered authoritative events plus rejection reasons.
replay_to(sequence)	Reconstructed state or first divergence record.
render_preview / export_gltf / export_usda	Non-authoritative presentation artifacts.

4. Scene, asset and instance composition

A game/graphics substrate needs stable content and scene structure in addition to mathematical primitives.

4.1 Asset contract

An asset is an immutable or content-addressed payload containing a derived mesh, material binding, source-pattern reference, schema hash and compilation metadata. The reference content hash is SHA-256 over canonical geometry and metadata. It is used for cache keys, package integrity and peer negotiation; it is not a substitute for full state, provenance or legal authorship.

4.2 Node and hierarchy contract

A scene node carries stable instance identity, optional asset binding, parent, local transform, visibility, tags, layer and lineage. World transforms use parent-first composition. Every parent edit runs cycle detection before it is accepted. This makes editor hierarchy operations ordinary guarded state changes rather than hidden recursive behavior.

```
T_world(node) = T_world(parent(node)) * T_local(node)

asset identity   != node identity
node coordinate  != lineage identity
render mesh      != authoritative relation definition
```

4.3 Layers and variants

The reference layer model applies ordered field opinions over a base node. It demonstrates non-destructive authoring without attempting to reproduce a full USD composition engine. A production adapter may map these opinions to USD layers, references, payloads and variant sets while preserving UGTS node IDs and event lineage.

4.4 Schema, units and migration

Required metadata	Purpose
schema_version	Selects field meanings, bit layouts and migration path.
units_per_meter	Converts authored linear quantities without implicit engine defaults.
up_axis / handedness	Prevents silent orientation changes across adapters.
time_unit / fixed_dt	Separates authored time, simulation ticks and presentation interpolation.
working_color_space	Makes preview and export color transforms reproducible.
determinism_profile	Declares bit-exact, tolerance-certified or presentation-only behavior.
migration_history	Records explicit from/to upgrades and operations.

4.5 Scene graph and spatial relationship

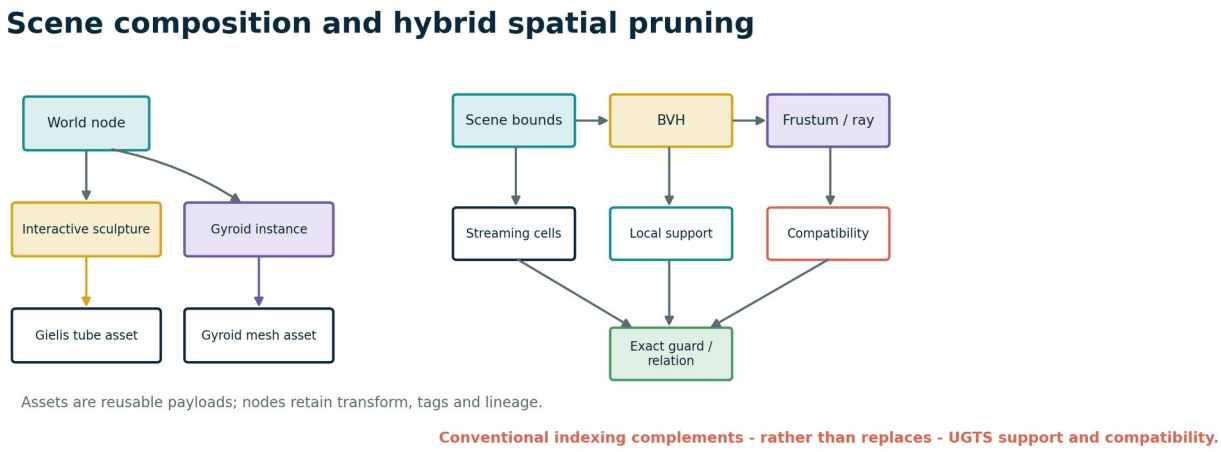


Figure 4. Assets and nodes form the composition graph. Bounds and BVH provide conventional indexing before local support, compatibility and exact guard work.

5. Geometry compiler and approximation contracts

Version 2.0 patterns become usable graphics assets without surrendering authority to a triangle mesh.

5.1 Adaptive parametric curves

The reference compiler flattens cubic Bezier curves using the maximum distance of the interior control points from the endpoint chord. Subdivision stops when the declared flatness tolerance is met or a depth limit is reached. The resulting polyline can feed a stroke, sweep, collision path, arc-length table or editor display.

```
flatness = max(distance(P1, line(P0,P3)),
               distance(P2, line(P0,P3)))
accept segment when flatness <= tolerance or depth == max_depth
```

5.2 Sweeps and moving frames

A tube mesh is built by sweeping a circular profile along a sampled 3D path. Parallel-transport frames are used so the profile remains stable when curvature approaches zero. Vertices receive angular and cumulative-length UV coordinates. The reference output is deterministic and directly exportable.

```
v(i,j) = p_i + r * (cos(theta_j) * N_i + sin(theta_j) * B_i)
N_i = normalize(N_(i-1) - T_i * dot(N_(i-1), T_i))
B_i = normalize(T_i x N_i)
```

5.3 Implicit fields to meshes

Bounded field meshing samples a declared lattice and decomposes each cube into tetrahedra. Crossing edges are interpolated and triangulated. Analytic gradients are preferred for normals; the reference uses centered finite differences when only a scalar field is available. Resolution, bounds and iso value are explicit. There is no hidden infinite raymarch loop.

5.4 Independent error budgets

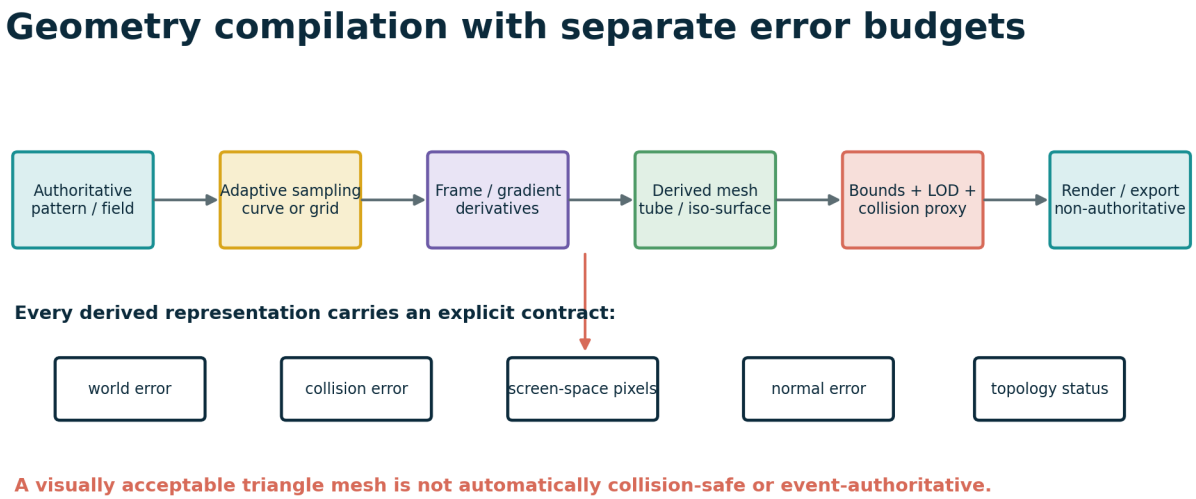


Figure 5. The source pattern or field stays authoritative. Derived render, collision and LOD objects carry independent error and topology status.

Budget	Question answered
world_error	How far may the derived surface move from the declared source geometry?
collision_error	What positional error is acceptable for broad/narrow phase?
screen_space_error_pixels	What silhouette or projection error is acceptable for a view?
normal_error_degrees	What lighting/frame orientation error is acceptable?
topology_preserved	Did compilation/simplification preserve required connectivity and components?

NON-NEGOTIABLE GEOMETRY BOUNDARY

A visually acceptable mesh is not automatically simulation-safe. A collision proxy, LOD or field mesh may be promoted only under an application-specific error and topology contract.

6. Spatial acceleration, culling and streaming

UGTS support predicates work best when combined with ordinary scene indexing rather than treated as a universal replacement.

6.1 AABB and BVH

The reference spatial layer provides finite axis-aligned bounds, unions, overlap, containment, transformed bounds and slab-based ray intervals. A deterministic BVH splits entries at the median center along the longest axis. This is intentionally conventional: it removes broad spatial irrelevance before support and compatibility perform their semantic filtering.

6.2 Hybrid candidate path

```
BVH / grid / frustum candidates
    -> support predicate
    -> compatibility predicate
    -> exact pattern / field / trajectory evaluation
    -> guard classification
    -> event proposal
```

6.3 View and editor queries

Frustum tests use the AABB positive vertex against each view plane. Ray queries return ordered bound hits for picking or coarse interaction. Every rejection can carry a reason code: outside frustum, outside support, incompatible, or visible. These reason codes are important for both editor inspection and performance analysis.

6.4 Streaming and interest management

Bounds enumerate regular-grid cells for a reference world partition. Per-user interest sets combine spatial overlap with required tags. A network adapter can therefore replicate schema, assets, checkpoints and events only for the relevant region while retaining the same authoritative sequence and lineage model.

6.5 Kill criterion

PERFORMANCE BOUNDARY

The hybrid design is rejected for a workload when BVH plus support and compatibility costs exceed a conventional mesh/indexing implementation at equal error, or when event/branch density removes the expected storage and query advantage.

7. Rendering, materials, color and interchange

Presentation becomes practical, replaceable and measurable without becoming state authority.

7.1 Typed preview materials

The reference PBR material records base color, metallic, roughness, emissive, alpha mode, cutoff and double-sided state. A small acyclic graph can read pattern context and combine constants, pattern values, add, multiply, mix and clamp nodes. The graph is deliberately presentation-only; it exposes no state mutation function.

7.2 Light and color pipeline

The CPU preview uses a compact directional Lambert-style shader for deterministic regression images. Scene-linear color may be exposed and passed through Reinhard or fitted ACES-like preview tone mapping before sRGB encoding. The scene schema can name OCIO and ACES targets, but the package does not bundle a production OCIO processor.

7.3 CPU/SVG reference renderer

The renderer transforms scene vertices, projects them through a declared camera, shades triangles and writes SVG. It exists to make the package inspectable and to generate regression artifacts. It is not a rasterization-performance claim and does not redefine the scene or event state.

7.4 Asset and scene interchange

Target	Reference output	Authority
glTF 2.0	Self-contained JSON with embedded binary buffer, mesh, nodes and PBR fields.	Runtime presentation asset; non-authoritative.
USDA	Readable Xform/Mesh hierarchy with UGTS custom metadata.	Authoring/interchange preview; full USD composition not claimed.
UGTS JSON	Scene, assets, nodes, hands, proposals and authority policy.	Authoritative scene definition when application policy says so.
Replay JSON	Initial scene, ordered events and checkpoints.	Authoritative event/lineage record.
SVG	Deterministic CPU preview.	Non-authoritative projection.

7.5 Backend targets

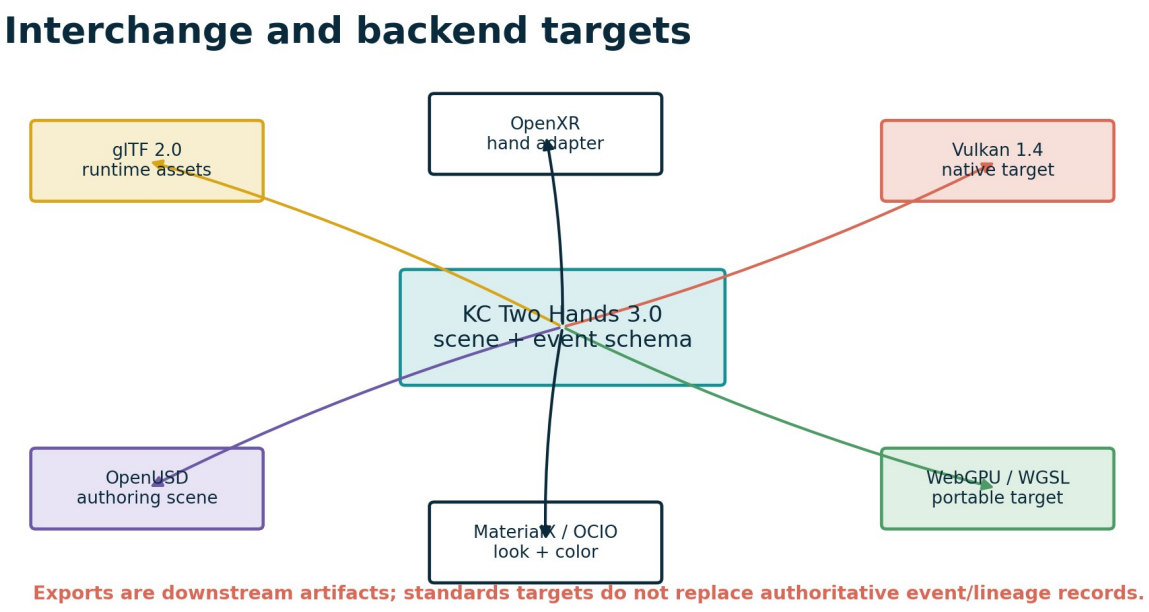


Figure 6. Open standards are adapter targets around a stable KC scene and event schema. The reference package implements only the documented subset.

The Vulkan profile is a physical native target, while WebGPU/WGSL is a portability target. The bundled shader files define instance and event-proposal layouts and a minimal preview pass. They are not compiled 3.0 SPIR-V modules and were not benchmarked on a named physical GPU.

8. Game runtime, animation and physics boundary

A practical game integrates conventional systems while preserving event and lineage authority.

8.1 Fixed-step runtime

KC Two Hands 3.0 separates the fixed simulation clock from input sampling and presentation. Systems may produce proposals during a tick; verification and deterministic commit occur before the tick advances. This produces stable sequence numbers, checkpoints and replay context without claiming cross-vendor bit identity for every floating-point operation.

8.2 ECS mapping

Component / system	Role
Transform + SceneNode	Instance pose, parent, visibility, tags and lineage reference.
Asset + Material	Derived geometry payload and presentation binding.
Support + Bounds	Local semantic admission and conventional spatial pruning.
Compatibility	Sheet, phase, mode, policy, ownership and application tags.
Motion / Kinematics	Closed-form state_at, moving frames or bounded integration.
Guard / Relation	Impact, portal, trigger, interaction or animation event condition.
Constraint	Resting contact, joint, grasp or maintained relation over an interval.
Event / Replay	Ordered patch, pre/post hashes, lineage and correction history.

8.3 Impact versus persistent constraints

```
impact or portal crossing  -> discrete event
resting contact interval   -> maintained constraint
joint or cooperative grab  -> maintained constraint
constraint release         -> discrete event
coincident guard interval  -> constraint mode, not infinite events
```

This division extends the 2.0 tangency/coincidence policy. It prevents a contact manifold or two-hand grab from being represented as an unbounded sequence of zero crossings.

8.4 Animation and physics scope

The catalog defines skeletal hierarchy, animation-state and character-sweep contracts and retains the 2.0 damped least-squares IK operator. The package does not bundle a full skeleton player, rigid-body contact solver or character controller. Unsupported cases are explicitly routed to conventional engine adapters rather than hidden behind topological vocabulary.

9. KC Two Hands interaction calculus

The release name becomes a concrete bimanual input and ownership subsystem.

9.1 Typed hand state

```
HandPose = (side, wrist_position, wrist_orientation,
            joint_positions, joint_radii,
            confidence, tracked, source, timestamp)
```

Left and right are declared schema values. The same structure can be populated by OpenXR hand joints, optical tracking, gloves, controllers, motion capture or deterministic desktop emulation. Device semantics stay in the adapter. Application changes remain gated events.

9.2 Pinch guard

```
inactive -> active when thumb_index_distance <= enter_distance
active   -> inactive when thumb_index_distance >= exit_distance
require exit_distance > enter_distance
```

The reference detector returns pinch-enter, pinch-exit, hold, missing-joint or tracking-unavailable reason codes. Low confidence resets the reference state; a production application may choose freeze or soften, but the policy must be explicit and replayed.

9.3 Bimanual transform

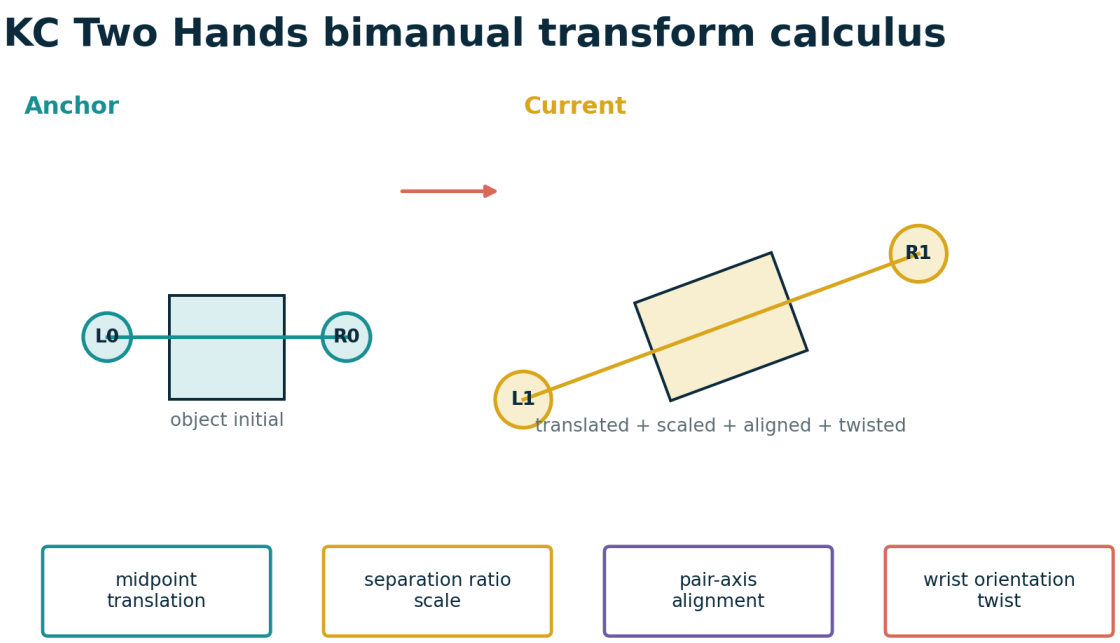


Figure 7. Pair midpoint, separation, pair-axis alignment and wrist twist jointly define the two-hand transform. Degenerate or low-confidence input returns a bounded rejection state.

```
midpoint m       = (p_left + p_right) / 2
scale s          = clamp(||right-left|| / ||right0-left0||)
alignment q_a    = shortest rotation mapping initial pair axis to current pair axis
twist q_t        = averaged wrist-orientation twist around the current pair axis
rotation q       = q_t * q_a
world delta      = T(m_current) * R(q) * S(s) * T(-m_anchor)
```

9.4 Ownership and handover

The cooperative-grab state machine distinguishes no owner, one-hand owner and two-hand owner. A second hand joining emits a separate event. When one hand releases a two-hand object, the remaining hand keeps ownership and a handover lineage label is appended. Object identity and scene-node identity do not change.

9.5 Interaction event path

```
tracked joint poses
    -> hand support volumes
    -> object / policy compatibility
    -> pinch or grab guard with hysteresis
    -> verified interaction proposal
    -> deterministic transform / ownership patch
    -> lineage + replay record
```

10. Event proposal, replay and networking

Parallel detection is separated from authoritative state mutation.

10.1 Proposal verification

```
verified = support_ok
       and compatibility_ok
       and guard_status in accepted_statuses
       and confidence >= confidence_floor
       and numeric_error <= event_margin
```

A proposal records ID, event time, type, entity, payload, source, priority, confidence, guard status, numerical error, event margin and lineage label. GPU kernels, input adapters and CPU systems can share this record. Gameplay-critical GPU output is a proposal, not an unchecked state write.

10.2 Deterministic commit and conflicts

Verified proposals are ordered by event time, descending priority, source and proposal ID. Proposals that request incompatible changes to the same entity field at the same event time are resolved by the first ordered winner and the remainder receive explicit conflict rejections. A production system may add commutative merges, but the merge policy must be versioned.

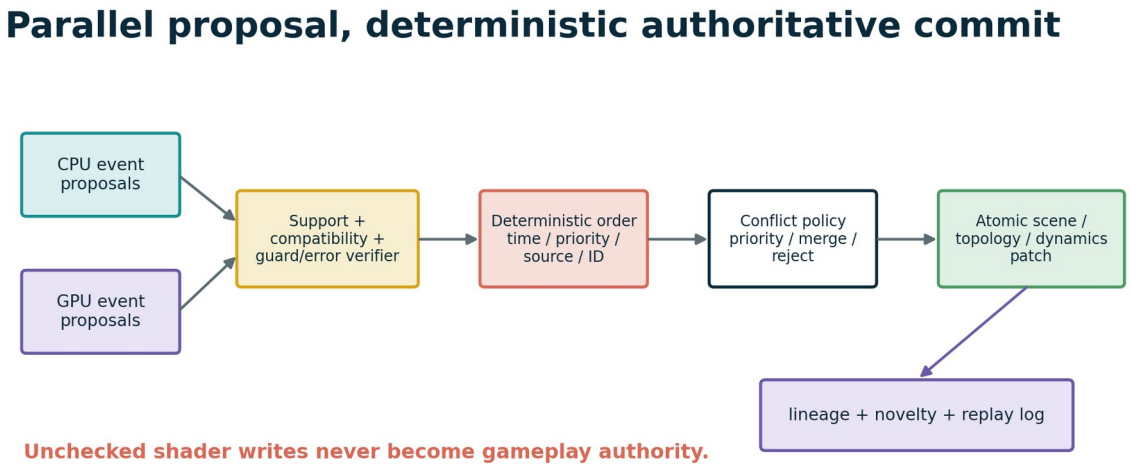


Figure 8. CPU and GPU systems may detect in parallel. Verification, ordering, conflict policy, patching and lineage remain a single authoritative boundary.

10.3 State hashes and checkpoints

Every committed event carries a monotonic sequence and pre/post state hashes. Checkpoints hash a runtime snapshot. Replay begins from the initial scene or nearest checkpoint, applies events in order and stops at the first hash mismatch. The digest is a divergence check; it is never treated as a replacement for full state or lineage.

10.4 Networking profile

- Server-authoritative or designated-authority event commit.
- Schema version and required mechanism negotiation.
- Asset content-hash availability before authoritative replay.
- Interest sets derived from spatial bounds, local support and tags.
- Periodic correction checkpoints and rollback.
- Restricted bit-exact or tolerance-certified numerical profiles.
- Explicit external input and nondeterministic novelty records.

DETERMINISM BOUNDARY

The package demonstrates deterministic ordering and hash-checked replay in one reference environment. It does not claim universal bit-identical floating-point execution across all CPUs, GPUs and compilers.

11. Runnable sandbox vertical slice

One demonstrator exercises geometry, scene, hands, events, replay, rendering and export together.

11.1 Scenario

1. Sample a Gielis pattern and compile it to a swept 3D tube using parallel-transport frames.
2. Sample a gyroid field in bounded space and compile an iso-surface through marching tetrahedra.
3. Create reusable assets, PBR materials and two scene instances under a world node.
4. Render a deterministic SVG preview before interaction.
5. Create a left/right bimanual anchor and evaluate a translated, scaled, aligned and twisted hand pair.
6. Emit and verify a set-transform proposal, commit it authoritatively and append lineage.
7. Render the post-event preview, export glTF and USDA, and write scene, replay and diagnostic JSON.

Figure 9. The compiled Gielis tube is transformed by one authoritative two-hand event. The gyroid asset remains stationary. Both panels are downstream SVG previews.

11.2 Captured values

Metric	Captured value
Bimanual status	ok
Translation	(0.325, 0.8, 0.225)
Scale	1.674136904
Twist	0.197877141 rad
Confidence	1.000
Committed events	1
Visible nodes	2
Rendered triangles	1968
Scene hash	e061c325aaacfa32da19743dc5215067cdbd183ceda9bcfbdc80483b904f0b22

11.3 Output artifacts

File	Purpose
sandbox_before.svg / sandbox_after.svg	Deterministic reference projections.
sandbox_scene.gltf	Self-contained runtime mesh/material/scene asset.

File	Purpose
sandbox_scene.usda	Readable authoring-oriented scene hierarchy.
sandbox_scene.json	Full embedded scene and geometry definition.
sandbox_replay.json	Initial scene, checkpoint and authoritative event.
sandbox_diagnostics.json	Event timeline, lineage edges and metrics.
sandbox_scene_audit.json	Scene integrity and bounds audit.
sandbox_summary.json	Compact captured run summary.

12. Validation, acceptance and kill criteria

The package establishes bounded functional consistency, not universal production performance.

12.1 Executable verification

VALIDATION RESULT

Python 3.13.5 executed 117 unit tests in the captured run. Status: PASS. The retained 2.0 suite covers pattern, field, topology, kinematics, dynamics and event behavior; the 3.0 suite adds scene, geometry compilation, spatial queries, materials, hand interaction, runtime, replay, render and export tests.

Validation layer	Status	What is established
KC 2.0 retained tests	47 PASS	Baseline math and event implementation remains internally consistent.
KC 3.0 new tests	70 PASS	Reference production-layer functions satisfy included properties.
JSON Schema example	PASS	Bundled example validates against the 3.0 schema.
Sandbox execution	PASS	All documented outputs were regenerated.
Catalog traceability	PASS	M258-M329 contiguous; counts check to 72 and total 329.
glTF structural checks	PASS in tests	Version, embedded buffer length and node/mesh references checked.
USDA / SVG structural checks	PASS in tests	Expected scene and projection structures emitted.
Physical-GPU benchmark	NOT CLAIMED	No named physical GPU run for the 3.0 renderer.
Full external-standard conformance	NOT CLAIMED	Targets are documented adapters, not complete implementations.

12.2 Numerical and geometry kill criteria

- Curve flatness, field resolution or normal error exceeds the declared event or visual margin.
- Marching output changes required topology or a lower-detail representation crosses an authoritative guard incorrectly.
- Bimanual separation becomes degenerate, confidence falls below policy, or gesture hysteresis chatters beyond budget.
- FP16/fixed-point/GPU error changes event ordering or exceeds the guard margin.
- Persistent contact is misrepresented as an unbounded coincident-event stream.

12.3 Performance and production kill criteria

- BVH plus support/compatibility costs more than a conventional implementation at equal error.
- Pattern opcode divergence, mesh generation, compaction or resource barriers dominate frame time.

- Asset streaming, shader compilation, replay checkpoints or event logs exceed memory and latency budgets.
- Cross-platform replay diverges beyond the selected deterministic/tolerance profile.
- A conventional renderer, physics engine or editor workflow is cheaper, faster or more accurate for the target workload.
- A physical-GPU advantage is promoted without a named device, driver, thermal/power state, p50/p95/p99 latency and oracle comparison.

12.4 Reproduction

```
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python examples/two_hands_sandbox.py
python -m json.tool examples/kc_two_hands_definition.json
```

13. Final upgraded definition and roadmap

KC Two Hands 3.0 is a tested production companion around the retained query-first substrate.

13.1 Final definition

UGTS-KC Two Hands 3.0 defines a finite, queryable algebra over typed geometry, topology, kinematics and bounded dynamics, extended by stable scene composition, deterministic geometry compilation, conventional spatial indexing, presentation materials, typed bimanual interaction and replayable event commit. Assets and meshes are derived payloads. Nodes and lineage preserve identity. GPU and hand systems may detect in parallel, but discrete change remains event-authoritative.

```
KC3 = (G,P,F,K,D,R,S,C,T,I,L, A,X,M,V,H,N,E)

canonical shorthand:
pattern/field + kinematic state
 -> spatial/support pruning
 -> compatibility
 -> guard classification
 -> verified proposal
 -> deterministic commit
 -> scene/topology/dynamic patch
 -> lineage + replay/novelty log
 -> optional render/export
```

13.2 Release interpretation

STATUS

Technically coherent and executable as a reference vertical slice. Production Vulkan/WebGPU rendering, OpenXR binding, OpenUSD/MaterialX/OCIO integration, character animation/physics and multiplayer transport remain explicit next-stage engineering tasks.

13.3 Recommended next measurements

1. Port the core scene/event records to a native C++ or Rust implementation with a stable C ABI and Python oracle comparison.
2. Compile the 3.0 WGSL/GLSL contracts and run a named physical Vulkan GPU with p50/p95/p99 frame and proposal latency.
3. Implement topology-aware mesh simplification and error propagation from source pattern/field to collision and render LOD.
4. Bind OpenXR hand joints to `HandPose`, capture confidence/fallback behavior and benchmark end-to-end interaction latency.
5. Add skeletal animation, maintained constraints, character sweeps and conventional rigid-body fallback.
6. Exercise server-authoritative event replication, interest management, checkpoints and rollback under packet delay/loss.
7. Integrate OpenUSD composition, glTF packaging, MaterialX looks and OCIO transforms without duplicating authoritative semantics.

8. Compare the vertical slice with a conventional mesh/BVH/physics baseline at equal error and apply the documented kill criteria.

Appendix A. M258-M329 extended mechanism catalog

The complete definitions, formulas and integration notes are supplied in CSV/JSON. This appendix is the compact human-readable index.

ID	Domain	Mechanism	Validation
M258	Scene and assets	Stable asset identity	Implemented/tested
M259	Scene and assets	Content-addressed asset hash	Implemented/tested
M260	Scene and assets	Scene-node transform hierarchy	Implemented/tested
M261	Scene and assets	Hierarchy cycle rejection	Implemented/tested
M262	Scene and assets	Asset-instance separation	Implemented/tested
M263	Scene and assets	Layered scene opinions	Implemented/tested
M264	Scene and assets	Coordinate-system declaration	Implemented/tested
M265	Scene and assets	Unit and time-base declaration	Implemented/tested
M266	Scene and assets	Streaming-region membership	Implemented/tested
M267	Scene and assets	Schema migration record	Implemented/tested
M268	Geometry compilation	Adaptive cubic-curve tessellation	Implemented/tested
M269	Geometry compilation	Dual simulation and presentation error contract	Implemented/tested
M270	Geometry compilation	Even-odd and nonzero fill rules	Implemented/tested
M271	Geometry compilation	Stroke expansion	Implemented/tested
M272	Geometry compilation	Swept tube mesh	Implemented/tested
M273	Geometry compilation	Parallel-transport sweep frames	Implemented/tested
M274	Geometry compilation	Implicit-field grid sampling	Implemented/tested
M275	Geometry compilation	Marching-tetrahedra meshing	Implemented/tested
M276	Geometry compilation	Field-gradient normal generation	Implemented/tested
M277	Geometry compilation	Sweep UV generation	Implemented/tested
M278	Geometry compilation	Collision proxy generation	Implemented/tested
M279	Geometry compilation	Reference mesh LOD hierarchy	Implemented/tested (reference)
M280	Spatial and streaming	Axis-aligned bounding box contract	Implemented/tested
M281	Spatial and streaming	Oriented-bound proxy contract	Specified only
M282	Spatial and streaming	Deterministic BVH construction	Implemented/tested
M283	Spatial and streaming	Support-aware broad phase	Implemented/tested
M284	Spatial and streaming	Frustum visibility query	Implemented/tested
M285	Spatial and streaming	Ray and picking query	Implemented/tested
M286	Spatial and streaming	World streaming cells	Implemented/tested
M287	Spatial and streaming	Per-user interest set	Implemented/tested
M288	Spatial and streaming	Culling reason codes	Implemented/tested
M289	GPU and rendering	Pattern-opcode dispatch	Shader contract supplied
M290	GPU and rendering	Parameter-buffer reference	Shader contract supplied
M291	GPU and rendering	Derivative availability flags	Shader contract supplied
M292	GPU and rendering	Render-graph resource contract	Specified only
M293	GPU and rendering	GPU-driven draw-list contract	Specified only
M294	GPU and rendering	GPU event proposal buffer	Specified + runtime schema

ID	Domain	Mechanism	Validation
M295	GPU and rendering	Deterministic authoritative commit boundary	Implemented/tested
M296	GPU and rendering	Vulkan backend profile	Specified only
M297	GPU and rendering	WebGPU/WGSL portability profile	Prototype supplied
M298	GPU and rendering	CPU SVG reference renderer	Implemented/tested
M299	Materials and color	Typed metallic-roughness PBR material	Implemented/tested
M300	Materials and color	Procedural pattern coordinates	Implemented/tested
M301	Materials and color	Acyclic material graph	Implemented/tested
M302	Materials and color	Material authority boundary	Specified + enforced by API shape
M303	Materials and color	Light and shadow contract	Partially implemented
M304	Materials and color	HDR exposure and tone mapping	Implemented/tested
M305	Materials and color	OCIO and ACES metadata target	Metadata specified
M306	Materials and color	Transparency policy	Implemented/tested
M307	Game runtime and animation	Fixed simulation clock separation	Implemented/tested
M308	Game runtime and animation	ECS component mapping	Specified
M309	Game runtime and animation	Skeletal hierarchy contract	Specified
M310	Game runtime and animation	Animation state-machine contract	Specified
M311	Game runtime and animation	IK and maintained-constraint bridge	Inherited implementation + specified bridge
M312	Game runtime and animation	Conventional rigid-body fallback	Specified boundary
M313	Game runtime and animation	Impact versus persistent constraint split	Specified boundary
M314	Game runtime and animation	Character sweep-controller contract	Specified only
M315	Two-hand interaction	Typed hand state	Implemented/tested
M316	Two-hand interaction	Pinch hysteresis	Implemented/tested
M317	Two-hand interaction	Bimanual midpoint frame	Implemented/tested
M318	Two-hand interaction	Bimanual scale	Implemented/tested
M319	Two-hand interaction	Bimanual twist and alignment	Implemented/tested
M320	Two-hand interaction	Cooperative grab ownership	Implemented/tested
M321	Two-hand interaction	Handover lineage event	Implemented/tested
M322	Two-hand interaction	Tracking confidence and desktop fallback	Implemented/tested
M323	Networking, editor and validation	Authoritative event sequence	Implemented/tested
M324	Networking, editor and validation	Checkpoint and rollback	Implemented/tested
M325	Networking, editor and validation	Deterministic canonical state hash	Implemented/tested
M326	Networking, editor and validation	Schema and asset negotiation	Specified
M327	Networking, editor and validation	Editor diagnostics and inspector records	Implemented/tested
M328	Networking, editor and validation	Replay divergence detection	Implemented/tested
M329	Networking, editor and validation	Physical-GPU benchmark promotion gate	Acceptance gate; not claimed

Appendix B. API and exchange dictionary

Public reference modules and record fields.

B.1 Python modules

Module	Primary responsibility
math3d.py	Vector, quaternion and matrix conventions.
geometry.py	Curve tessellation, frames, tubes, strokes, field meshing, LOD and proxies.
spatial.py	AABB, BVH, ray/frustum queries, cells, interest and culling reasons.
scene.py	Assets, nodes, hierarchy, layers, bounds, serialization and content hashes.
materials.py	PBR data, preview color pipeline and acyclic material graph.
hands.py	Hand pose, pinch detector, bimanual transform and cooperative ownership.
runtime.py	Fixed-step systems, proposal verification, deterministic commit and snapshots.
replay.py	Checkpoints, rollback, event replay and divergence detection.
render.py	Deterministic CPU/SVG projection.
export.py	glTF, USDA and scene JSON adapters.
diagnostics.py	Scene audits, timelines, lineage edges and runtime reports.

B.2 Core record dictionary

Record	Required fields / meaning
Asset	id, mesh, material_id, source_pattern_id, schema_hash, metadata, content_hash.
SceneNode	id, asset_id, parent_id, local_transform, visible, tags, layer, lineage.
HandPose	side, wrist pose, joints/radii, confidence, tracked, source, timestamp.
EventProposal	ID, time, type, entity, payload, source, priority, gates, error, lineage label.
RuntimeEvent	sequence, event, payload, confidence, lineage, pre/post hash, runtime tick/time.
Checkpoint	sequence, runtime snapshot, snapshot hash.
GeometryErrorContract	world, collision, pixel and normal error plus topology status.
CullResult	item ID, visible flag and reason code.

B.3 Production instance and proposal layout

```
InstanceRecord:
  world transform | bounds | pattern opcode | parameter offset
  material index | derivative flags | support/compatibility masks
  lineage checksum | LOD policy

EventProposal:
  event time | guard value/status | numeric error | confidence
  entity | source | priority | requested patch | lineage label
```

The shader prototypes in `shaders/` express these ideas in WGSL and GLSL. The Python structures are the behavioral reference. A native ABI must freeze alignment, widths, endianness and schema hashes before cross-language or GPU exchange.

Appendix C. Source, standards and package verification

Reproducibility and scope register.

C.1 Source hashes

Role	Filename	SHA-256
UGTS-0 source architecture	Unified_Geometric_Topological_Substrate(1).pdf	c991aba24cc939ba680697b62b9e19545131cf34060793490bf149c6e2ab522e
UGTS-GN 1.1 GPU-native addendum	Unified_Geometric_Topological_Substrate_GPU_Native_Addendum(2).pdf	3a6b6c57118ca4e2d7d7e577508c3477a095d133791f3708cb56a054285225d2
UGTS-KC 2.0 immediate baseline package	UGTS_KC_2_0_Tom_Klootwijk_Package.zip	834997b0139ee43d366dbdddcb25116c7f62081e0ff7799ccf700c1257f12b6d
UGTS-KC 2.0 companion report	UGTS_KC_2_0_Tom_Klootwijk_Expanded_Substrate.pdf	a29dd9959ebd300c32d0e1df8fa2c0d4ee15f428892557b98b5f18cd3e878399

C.2 Standards targets checked 16 August 2026

Target	Reference version / role	Package status
glTF	2.0.1 runtime asset specification	Reference exporter subset.
OpenUSD	26.08 documentation / scene composition target	USDA text output; full runtime not bundled.
Vulkan	1.4 native graphics/compute target	Contract only; no 3.0 physical-GPU benchmark.
WebGPU / WGSL	Portable API and shader-language targets	WGSL prototype supplied.
OpenXR	1.1 and XR_EXT_hand_tracking input target	Schema target; no runtime call.
MaterialX	1.39 material interchange target	Independent compact graph only.
OpenColorIO / ACES	Color-management workflow target	Metadata and preview transforms only.

C.3 Package verification sequence

1. Copy the retained KC 2.0 Python oracle and 47-test suite.
2. Generate M258-M329 and machine-check contiguous IDs and counts.
3. Compile all Python modules.
4. Run all 117 tests and capture console output and summary JSON.
5. Validate the example definition against the bundled JSON Schema.
6. Execute the sandbox and regenerate SVG, glTF, USDA, scene, replay and diagnostics.
7. Regenerate report diagrams and inspect representative image outputs.
8. Render the DOCX to PDF and page images; inspect every page for clipping, overlap and broken tables.
9. Hash every delivered file and create the final ZIP manifest.

CLOSING BOUNDARY

The package proves a coherent, runnable reference design. It does not prove commercial readiness, external-standard completeness, physical-GPU advantage or legal ownership. Those require separate integration, measurement and review.